汎用データ解析API 次期計画書

1. 背景

レビュー指摘の要約

本番レビューにおいて、以下の提案を受けた。

1. **名称・コンセプトの拡張**: 「画像解析API」ではなく「データ解析API」として、汎用的なコンセプトで設計すべき
   - ドメインごとにAPIが乱立するより、汎用解析APIとして一つにまとめた方が管理しやすくクライアントも迷わない
   - 途中で画像解析→汎用解析に移行すると、パスパラメータのリソース関係やパラメータ配置に揺れが生じる
2. **APIドキュメントの事前レビュー**: 機能追加前に開発三課でAPI設計をレビューすべき
   - APIは変更できず追加しかできないインターフェース
   - コンセプトを把握した上で慎重に設計しないと無秩序になる

現モックの位置づけ

- 現在の `image-analysis` はPoC/モックとしてそのまま維持する
- `https://image-analysis.example-cloud.com` は既に組織内共有済みのため、デモとして引き続き稼働させる
- 本運用APIは新規に設計・構築する

---

2. コンセプト

汎用データ解析APIの方針

「指定されたデータソースを解析判定して回答するAPI」として設計する。

- 初期リリースは画像解析のみだが、API設計は複数のデータ型に対応できる構造とする
- 拡張候補: 画像、CSV/テーブルデータ、テキスト、センサーデータ(OSNET等)
- クライアントは単一のAPIエンドポイントに対して、データ型を指定して解析を依頼する

命名案

項目	現モック	次期API
プロジェクト名	image-analysis	data-analysis (仮)
サブドメイン	image-analysis.example-cloud.com	data-analysis.example-cloud.com (仮)
スタック名	image-analysis-api	data-analysis-api (仮)

※ 命名はレビューで確定する

---

3. API設計原則

レビューワーの指摘に基づく4原則を、本APIおよび今後のAPI開発の設計規約とする。

原則1: パスパラメータはリソースの相互関係を表す階層にする

```
良い例: /v1/analyses/{analysis_id}/results
悪い例: /api/analyze, /api/workflow/analyze (動詞ベース、関係性が不明)
```

原則2: パラメータの配置先に一貫した目的を持たせる

配置先	目的
パスパラメータ	リソースの識別・階層関係
クエリパラメータ	フィルタリング・ページング・ソート
ヘッダ	認証・メタ情報・コンテントネゴシエーション
リクエストボディ	リソースの作成・更新データ

原則3: リクエスト・レスポンスボディは適切に階層化する

```
// 悪い例（現モック）: キーが直列に並んでいる
{
  "image": "base64...",
  "image_url": "https://...",
  "image_bucket": "my-bucket",
  "image_key": "path/to/image.jpg",
  "target": "ひび割れ",
  "model": "gpt-4.1-mini"
}

// 良い例: 目的別に階層化
{
  "input": {
    "type": "image",
    "source": {
      "method": "s3",
      "bucket": "my-bucket",
      "key": "path/to/image.jpg"
    }
  },
  "analysis": {
    "target": "ひび割れ",
    "model": "gpt-4.1-mini",
    "detail": "auto"
  }
}
```

原則4: APIの後ろのプロセス群と整合性を取る

- StepFunctionsワークフローとの連携パターンを設計段階で考慮する
- 非同期処理が必要な場合は、ジョブ投入→ステータス確認→結果取得の3段階APIとする
- 内部プロセスの都合をAPIインターフェースに露出させない

---

4. API設計案（たたき台）

バージョニング

- URLパスにバージョンを含める: `/v1/...`
- 破壊的変更時は `/v2/...` を新設し、旧バージョンは廃止期限を設けて並行運用

リソース設計

```
POST   /v1/analyses              解析ジョブの作成（同期 or 非同期）
GET    /v1/analyses/{id}         解析結果の取得
GET    /v1/analyses              解析履歴の一覧（フィルタ・ページング）
GET    /v1/models                利用可能なモデル一覧
GET    /v1/health                ヘルスチェック
```

リクエストボディ（POST /v1/analyses）

```
{
  "input": {
    "type": "image",
    "source": {
      "method": "s3" | "url" | "base64",
      "bucket": "...",
      "key": "...",
      "url": "https://...",
      "data": "base64..."
    }
  },
  "analysis": {
    "target": "検出対象の説明",
    "model": "gpt-4.1-mini",
    "options": {
      "detail": "auto" | "low" | "high"
    }
  },
  "callback": {
    "url": "https://...",
    "method": "POST"
  }
}
```

レスポンスボディ

```
{
  "id": "ana_xxxxxxxxxxxx",
  "status": "completed",
  "input": {
    "type": "image",
    "source": { "method": "s3" }
  },
  "result": {
    "detected": true,
    "confidence": 0.95,
    "description": "解析結果の説明",
    "model": "gpt-4.1-mini",
    "tokens_used": 144
  },
  "metadata": {
    "created_at": "2026-03-05T12:00:00Z",
    "processing_time_ms": 11100
  }
}
```

※ 上記はたたき台であり、レビューで確定する

---

5. 移行方針

現モックの扱い

項目	対応
CloudFormationスタック	維持（image-analysis-api / image-analysis-dev）
サブドメイン	維持（image-analysis.example-cloud.com）
用途	組織内デモ・PoC検証用として継続利用
廃止時期	次期APIの本番リリース後に判断

新APIの構築

- 新規CloudFormationスタックとして構築（既存モックとは独立）
- 現モックで得た知見（ネストスタック、LambdaLogger、マルチ環境等）はそのまま活用
- テンプレート構成は現モックと同じパターン（親+api子+web子）を踏襲

---

6. 実施フロー

```
[Phase 1] 本計画書のレビュー
  ↳ 開発三課でAPI設計方針・リソース設計を議論
  ↳ 命名（プロジェクト名、サブドメイン）の確定
  ↳ 合意後、API仕様書（OpenAPI）のドラフト作成
```

```
[Phase 2] API仕様書レビュー
  ↳ OpenAPI定義の詳細レビュー（パス、パラメータ、ボディ構造）
  ↳ StepFunctionsワークフローとの連携パターン確認
  ↳ 合意後、実装開始

[Phase 3] 実装・テスト
  ↳ TDDで開発（テスト先行）
  ↳ dev環境デプロイ・動作確認
  ↳ prod環境デプロイ

[Phase 4] 現モック廃止判断
  ↳ 新APIで現モックの機能を包含していることを確認
  ↳ 利用者への周知・移行期間の設定
```

---

7. CLAUDE.mdへの反映（API設計憲法）

レビュー承認後、以下のルールをプロジェクトの `CLAUDE.md` に追記する。 これにより、今後のAPI開発で自動的にこれらの原則が適用される。

追記予定の内容

```markdown
## API Design Principles（API設計原則）

以下の4原則は全てのREST API設計に適用する。

### 1. パスパラメータの階層化
- パスパラメータはリソースの相互関係が伝わるように階層化する
- 動詞ではなく名詞（リソース）をパスに使う
- 例: `POST /v1/analyses`（解析ジョブ作成）、`GET /v1/analyses/{id}`（結果取得）

### 2. パラメータ配置の一貫性
- パス: リソースの識別・階層関係
- クエリ: フィルタリング・ページング・ソート
- ヘッダ: 認証・メタ情報
- ボディ: リソースの作成・更新データ
- API内でこの目的に揺れを生じさせない

### 3. リクエスト・レスポンスボディの階層化
- キーを直列に並べず、目的別に適切にネストする
- 入力データ、解析パラメータ、メタ情報等を分離する

### 4. バックエンドプロセスとの整合性
- APIインターフェースに内部実装の都合を露出させない
- 非同期処理はジョブ投入→ステータス確認→結果取得の3段階で設計する
- ワークフロー連携パターンを設計段階で考慮する

### 運用ルール
- 新規API・既存APIへの機能追加前に、API仕様書（OpenAPI）を作成しレビューを実施する
- APIは原則「追加のみ」。既存エンドポイントの破壊的変更は行わない
- バージョニングはURLパス方式（/v1/, /v2/）を採用する
```

反映先

- `/path/to/CLAUDE.md`（プロジェクト共通） → 全APIプロジェクトに適用される
- 反映タイミング: Phase 1のレビュー承認後

改訂履歴

日付	内容
2026-03-06	初版作成